

-Kabir Sinha

-E23CSEU1343

CSET419 – Introduction to Generative AI

Lab 5: CIFAR-10 Image Reconstruction using Convolutional Autoencoder

```
[1]
✓ 14s !pip -q install torch torchvision matplotlib

import torch
import torch.nn as nn
import torch.optim as optim
import torchvision
import torchvision.transforms as transforms
import matplotlib.pyplot as plt
import numpy as np
import os

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print(device)
```

... cuda

[2]

✓ 19s

```
transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.5, 0.5, 0.5),
                          (0.5, 0.5, 0.5))
])

train_dataset = torchvision.datasets.CIFAR10(
    root="./data",
    train=True,
    transform=transform,
    download=True
)

train_loader = torch.utils.data.DataLoader(
    train_dataset,
    batch_size=128,
    shuffle=True
)

print("Train samples:", len(train_dataset))
```

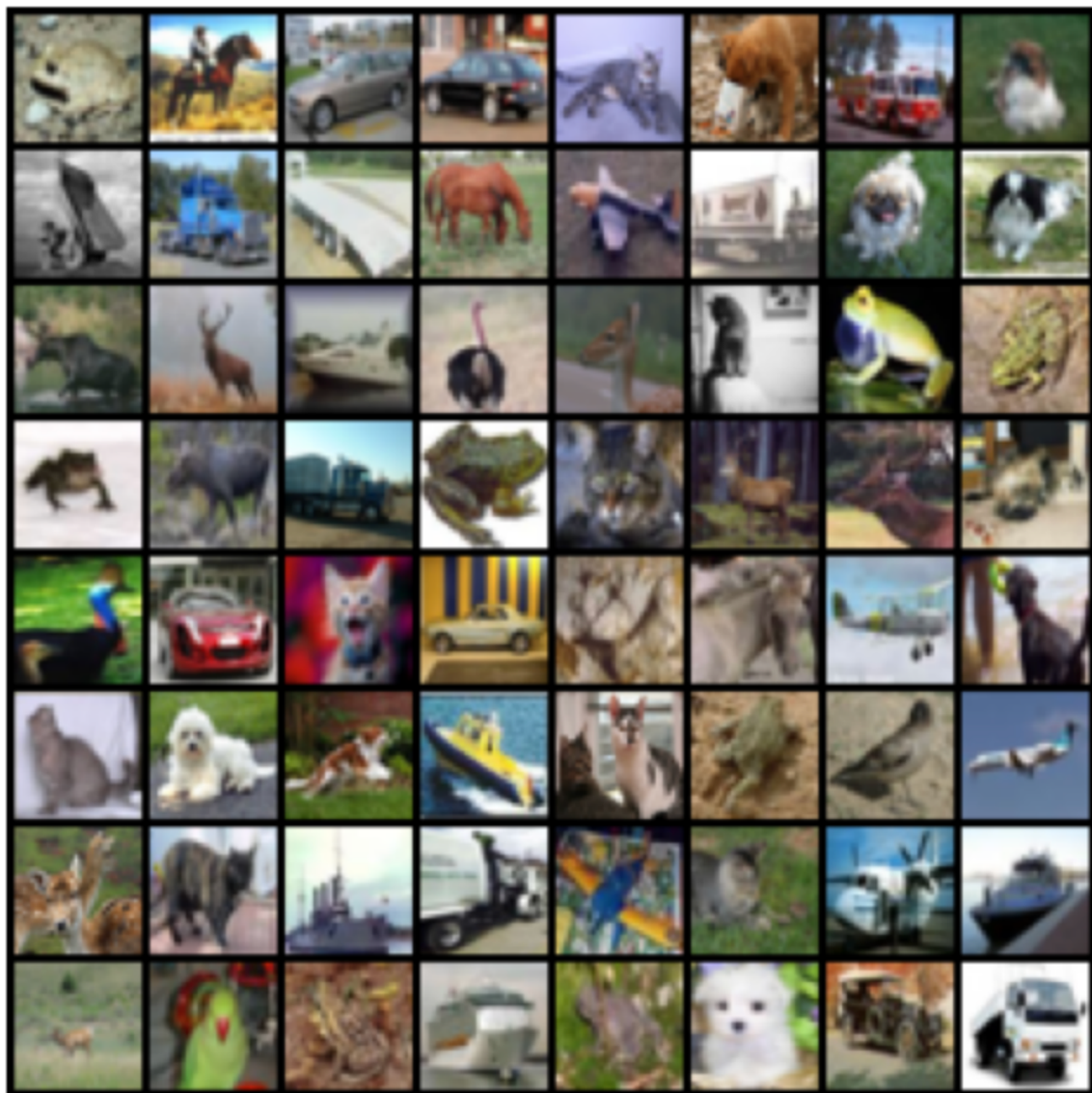
... 100%|██████████| 170M/170M [00:13<00:00, 12.7MB/s]
Train samples: 50000

[3]

✓ 7s

```
images, _ = next(iter(train_loader))

plt.figure(figsize=(6,6))
grid = torchvision.utils.make_grid(images[:64], nrow=8, normalize=True)
plt.imshow(np.transpose(grid.cpu(), (1,2,0)))
plt.axis("off")
plt.show()
```



[4]

✓ Os



```
class EncoderDecoder(nn.Module):
    def __init__(self):
        super().__init__()

        self.encoder = nn.Sequential(
            nn.Conv2d(3, 64, 4, 2, 1),
            nn.ReLU(True),

            nn.Conv2d(64, 128, 4, 2, 1),
            nn.ReLU(True),

            nn.Conv2d(128, 256, 4, 2, 1),
            nn.ReLU(True)
        )

        self.decoder = nn.Sequential(
            nn.ConvTranspose2d(256, 128, 4, 2, 1),
            nn.ReLU(True),

            nn.ConvTranspose2d(128, 64, 4, 2, 1),
            nn.ReLU(True),

            nn.ConvTranspose2d(64, 3, 4, 2, 1),
            nn.Tanh()
        )

    def forward(self, x):
        x = self.encoder(x)
        x = self.decoder(x)
        return x

model = EncoderDecoder().to(device)
```

[5]
✓ 0s

```
criterion = nn.L1Loss()
optimizer = optim.Adam(model.parameters(), lr=0.0002)

epochs = 15
train_losses = []
```

[6]
✓ 4m



```
model.train()

for epoch in range(epochs):
    total_loss = 0

    for images, _ in train_loader:
        images = images.to(device)

        optimizer.zero_grad()
        outputs = model(images)

        loss = criterion(outputs, images)
        loss.backward()
        optimizer.step()

    total_loss += loss.item()

avg_loss = total_loss / len(train_loader)
train_losses.append(avg_loss)

print(f"Epoch [{epoch+1}/{epochs}] Loss: {avg_loss:.4f}")
```

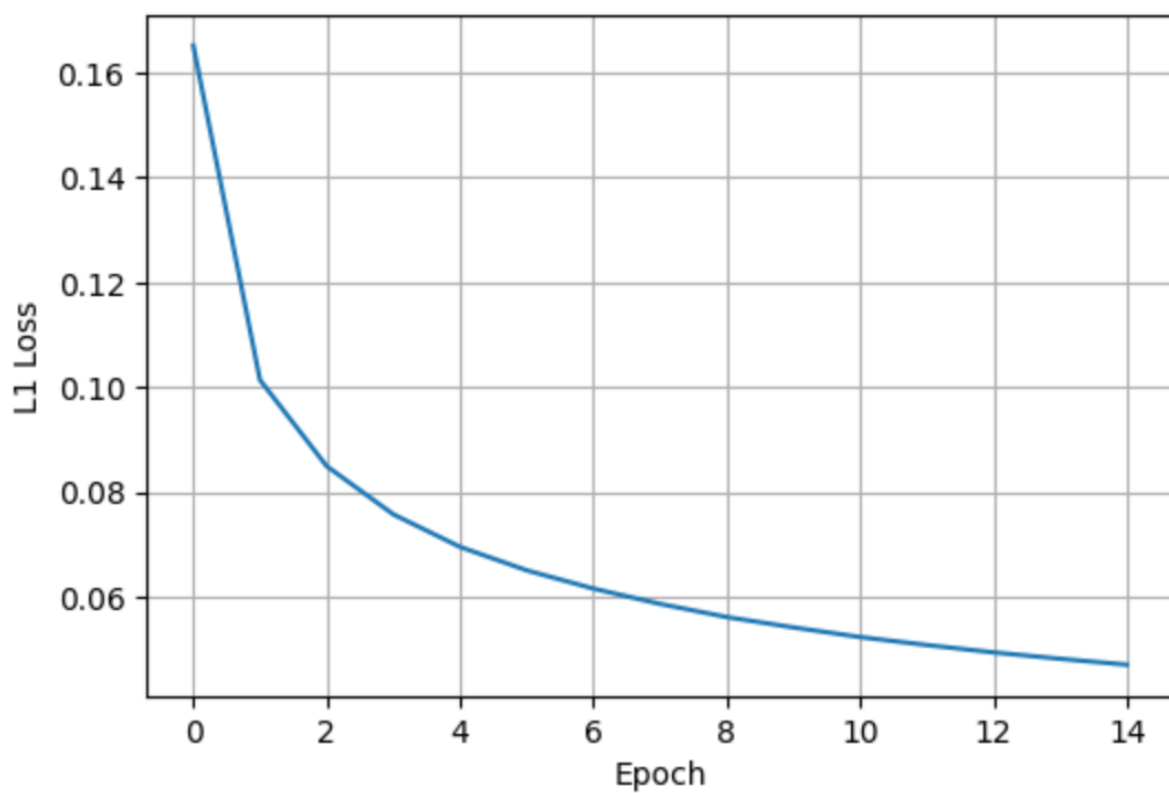


```
... Epoch [1/15] Loss: 0.1652
Epoch [2/15] Loss: 0.1014
Epoch [3/15] Loss: 0.0849
Epoch [4/15] Loss: 0.0757
Epoch [5/15] Loss: 0.0695
Epoch [6/15] Loss: 0.0650
Epoch [7/15] Loss: 0.0616
Epoch [8/15] Loss: 0.0586
Epoch [9/15] Loss: 0.0561
Epoch [10/15] Loss: 0.0542
Epoch [11/15] Loss: 0.0523
Epoch [12/15] Loss: 0.0508
Epoch [13/15] Loss: 0.0494
Epoch [14/15] Loss: 0.0482
Epoch [15/15] Loss: 0.0470
```

[7]

✓ 0s

```
plt.figure(figsize=(6,4))  
plt.plot(train_losses)  
plt.xlabel("Epoch")  
plt.ylabel("L1 Loss")  
plt.grid(True)  
plt.savefig("baseline_loss_curve.png", dpi=300, bbox_inches="tight")  
plt.show()
```



[8]

✓ 45s

```
model.eval()

with torch.no_grad():
    images, _ = next(iter(train_loader))
    images = images.to(device)
    outputs = model(images)

    comparison = torch.cat([images[:32], outputs[:32]])

plt.figure(figsize=(6,6))
grid = torchvision.utils.make_grid(comparison.cpu(), nrow=8, normalize=True)
plt.imshow(np.transpose(grid, (1,2,0)))
plt.axis("off")
plt.savefig("baseline_translation_results.png", dpi=300, bbox_inches="tight")
plt.show()
```



```
!pip -q install torch torchvision matplotlib
```

```
import torch
```

```
import torch.nn as nn
```

```
import torch.optim as optim
```

```
import torchvision
```

```
import torchvision.transforms as transforms
```

```
import matplotlib.pyplot as plt
```

```
import numpy as np
```

```
import os
```

```
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
```

```
print(device)
```

```
transform = transforms.Compose([  
    transforms.ToTensor(),  
    transforms.Normalize((0.5, 0.5, 0.5),  
                          (0.5, 0.5, 0.5))  
])
```

```
train_dataset = torchvision.datasets.CIFAR10(  
    root="./data",  
    train=True,  
    transform=transform,  
    download=True  
)
```

```
train_loader = torch.utils.data.DataLoader(  
    train_dataset,  
    batch_size=128,  
    shuffle=True  
)
```

```
images, _ = next(iter(train_loader))
```

```
plt.figure(figsize=(6,6))  
grid = torchvision.utils.make_grid(images[:64], nrow=8, normalize=True)  
plt.imshow(np.transpose(grid.cpu(), (1,2,0)))  
plt.axis("off")  
plt.show()
```

```
class EncoderDecoder(nn.Module):  
    def __init__(self):  
        super().__init__()  
  
        self.encoder = nn.Sequential(  
            nn.Conv2d(3, 64, 4, 2, 1),  
            nn.ReLU(True),  
  
            nn.Conv2d(64, 128, 4, 2, 1),  
            nn.ReLU(True),
```

```
nn.Conv2d(128, 256, 4, 2, 1),  
nn.ReLU(True)  
)
```

```
self.decoder = nn.Sequential(  
nn.ConvTranspose2d(256, 128, 4, 2, 1),  
nn.ReLU(True),
```

```
nn.ConvTranspose2d(128, 64, 4, 2, 1),  
nn.ReLU(True),
```

```
nn.ConvTranspose2d(64, 3, 4, 2, 1),  
nn.Tanh()  
)
```

```
def forward(self, x):  
    x = self.encoder(x)  
    x = self.decoder(x)  
    return x
```

```
model = EncoderDecoder().to(device)
```

```
criterion = nn.L1Loss()
```

```
optimizer = optim.Adam(model.parameters(), lr=0.0002)
```

```
epochs = 15
```

```
train_losses = []
```

```
model.train()
```

```
for epoch in range(epochs):
```

```
    total_loss = 0
```

```
    for images, _ in train_loader:
```

```
        images = images.to(device)
```

```
        optimizer.zero_grad()
```

```
        outputs = model(images)
```

```
        loss = criterion(outputs, images)
```

```
        loss.backward()
```

```
        optimizer.step()
```

```
    total_loss += loss.item()
```

```
avg_loss = total_loss / len(train_loader)
```

```
train_losses.append(avg_loss)
```

```
print(f"Epoch [{epoch+1}/{epochs}] Loss: {avg_loss:.4f}")
```

```
plt.figure(figsize=(6,4))
```

```
plt.plot(train_losses)
```

```
plt.xlabel("Epoch")
```

```
plt.ylabel("L1 Loss")
plt.grid(True)
plt.savefig("baseline_loss_curve.png", dpi=300, bbox_inches="tight")
plt.show()
```

```
model.eval()
```

```
with torch.no_grad():
    images, _ = next(iter(train_loader))
    images = images.to(device)
    outputs = model(images)
```

```
comparison = torch.cat([images[:32], outputs[:32]])
```

```
plt.figure(figsize=(6,6))
grid = torchvision.utils.make_grid(comparison.cpu(), nrow=8, normalize=True)
plt.imshow(np.transpose(grid, (1,2,0)))
plt.axis("off")
plt.savefig("baseline_translation_results.png", dpi=300, bbox_inches="tight")
plt.show()
```

```
torch.save(model.state_dict(), "baseline_encoder_decoder.pth")
print("Model saved.")
```